AJ SMART BIZ · DOCUMENT 10

Developer Guide & Coding Standards

Folder structure, code formats and conventions of every application, and the end-to-end recipe for adding a feature.

Audience	Developers joining the project, code reviewers, AI coding agents
Version / date	1.0 · 7 November 2026
AI skill	The same rules are packaged as the Claude Code skill `.claude/skills/aj-smart-biz-feature`

Contents

1. Repository layout
2. Golden rules
3. Backend — structure and conventions
4. Angular consoles — structure and conventions
5. Next.js websites — structure and conventions
6. Recipe: add a new feature end to end
7. Formatting, naming and comments
8. Definition of done / review checklist

1. Repository layout

```
AJ SmartBiz/
├─ Aj-Smart-Biz-Backend/        Express API (CommonJS)          :4000
│  ├─ src/{config,constants,models,validators,middlewares,controllers,
│  │       services,routes,modules,seeders,scripts,docs,utils}
│  └─ docs/schema.dbml
├─ Aj-Smart-Biz-Supper-Admin/   Angular 22 platform console     :4200
├─ Aj-Smart-Biz-Admin/          Angular 22 company workspace    :4300
│  └─ src/app/{core/{guards,interceptors,models,services},layout,shared/ui,features/*}
├─ themes/{default,midnight,atelier}/  Next.js 15 tenant sites  :4400/4500/4600
│  └─ src/{middleware.ts,app/{actions,api,uploads,...routes},components,config,lib}
├─ Aj-Smart-Biz-Technology/     Next.js 15 product website      :4700
├─ deploy/                      per-service env examples
├─ docs/                        this documentation (source + pdf)
└─ README.md                    product behaviour, in depth
```

2. Golden rules

1. **API-first, no hard-coded data.** Anything a client shows comes from a backend endpoint; templates never invent tenant content.
2. **Constants are the catalogue.** Enums, feature keys, transition maps and limits live in `Aj-Smart-Biz-Backend/src/constants/index.js`. Never duplicate a list elsewhere — mirror it in TypeScript types with a "Mirrors X in the API" comment.
3. **One decision, one function.** Entitlement (`functionality.service`), quota (`quota.service`), service state, offer status (`isOnOffer`), enquiry target — reuse them; never re-derive in a controller or UI.
4. **Guard at the module.** Add routes inside the right module (`/admin`, `/super-admin`, `/theme`, `/website`); never add ad-hoc auth.
5. **Tenant from token.** Use `resolveCompanyId(req)`; never accept `companyId` from an admin's body.
6. **Server-side money.** Prices, totals, discounts computed on the server.
7. **Snapshots for history; soft delete for everything else.**
8. **Explain why.** The codebase uses long comments that state business reasons and rejected alternatives — keep that style.
9. **Themes are triplets.** Logic changes in `themes/` are copied verbatim to all three designs; only CSS differs.

3. Backend

3.1 Folder responsibilities

Folder	Put here	File name
constants/index.js	Enums (`X` + `X_VALUES`), `X_CATALOGUE` , transitions, defaults	single file
models/	Sequelize factory per table; associations only in `index.js`	`companyWarehouse.model.js`
validators/	Joi `{params, query, body}` schemas; primitives in `common.js`	`warehouse.validator.js`
services/	Business rules, transactions, state machines	`stock.service.js`
controllers/	HTTP handlers with `asyncHandler`	`warehouse.controller.js`
routes/	Resource routers (`mergeParams: true`)	`warehouse.routes.js`
modules/	Composed router per consumer + guard	`admin/admin.routes.js`
seeders/	Idempotent boot steps, default menus	`bootstrap.js` , `defaultMenus.js`

3.2 Model

```
'use strict';
const { DataTypes } = require('sequelize');
const { STATUS, STATUS_VALUES } = require('../constants');

/** Why this table exists, what it is not, and the rules it keeps. */
module.exports = (sequelize) =>
  sequelize.define(
    'CompanyWarehouse',
    {
      id: { type: DataTypes.INTEGER.UNSIGNED, primaryKey: true, autoIncrement: true },
      companyId: { type: DataTypes.INTEGER.UNSIGNED, allowNull: false },
      branchId: { type: DataTypes.INTEGER.UNSIGNED, allowNull: true },
      name: { type: DataTypes.STRING(150), allowNull: false },
      sequence: { type: DataTypes.INTEGER, allowNull: false, defaultValue: 0 },
      status: { type: DataTypes.ENUM(...STATUS_VALUES), allowNull: false, defaultValue: STATUS.ACTIVE },
      createdBy: { type: DataTypes.INTEGER.UNSIGNED, allowNull: true },
      updatedBy: { type: DataTypes.INTEGER.UNSIGNED, allowNull: true },
    },
    { tableName: 'company_warehouses', indexes: [{ fields: ['company_id'] }, { fields: ['status'] }] }
  );
```

Register in `models/index.js` : require, associate (`Company.hasMany(X, { foreignKey: 'companyId', as: '…', onDelete: 'CASCADE' })` + `X.belongsTo(Company)`), and add to the exported `db` . Use SET NULL / `constraints:false` for history references. No unique indexes on business keys — check in the service.

3.3 Validator

```
const fields = {
  name: Joi.string().trim().max(150),
  code: Joi.string().trim().uppercase().max(30),
  branchId: id.allow(null),
  status,
};
const create = { body: Joi.object({ ...fields, name: fields.name.required(), code: fields.code.required()
}) };
const update = { params: idParam, body: Joi.object(fields).min(1) };   // no defaults on update
const list   = { query: listQuery({ branchId: Joi.alternatives().try(id, Joi.string().valid('none')) })
};
module.exports = { create, update, list };
```

3.4 Controller

```
const list = asyncHandler(async (req, res) => {
  const companyId = resolveCompanyId(req);
  const { page, limit, offset } = getPagination(req.query);
  const where = mergeWhere({ companyId },
    req.query.status ? { status: req.query.status } : null,
    buildSearch(req.query.search, ['name', 'code']));
  const result = await db.CompanyWarehouse.findAndCountAll({
    where, order: getSort(req.query, ['name', 'created_at'], [['sequence', 'ASC']]),
    limit, offset, distinct: true });
  return paginated(res, result, { page, limit }, 'Warehouse list fetched successfully');
});

const create = asyncHandler(async (req, res) => {
  const companyId = resolveCompanyId(req);
  await functionalityService.assertGranted(companyId, FUNCTIONALITY.WAREHOUSE);
  const row = await db.CompanyWarehouse.create({ ...req.body, companyId, createdBy: req.auth.id });
  return created(res, 'Warehouse created successfully', row);
});
```

- Errors: `throw ApiError.badRequest('Readable reason', errors?)`, `.notFound`, `.forbidden`, `.conflict`.
- Responses: `success()`, `created()`, `paginated()` — messages "<Label> created successfully".
- Ordered card lists: use `cardResource.factory.js`; platform masters: `crud.factory.js`.

3.5 Service

```
/** Always call inside a Sequelize transaction. */
async function applyMovement({ companyId, warehouseId, productId, reason, quantity }, { transaction }) {
  const rule = STOCK_REASON_CATALOGUE.find((r) => r.key === reason);
  if (!rule) throw ApiError.badRequest('Unknown stock reason');
  // ... read level FOR UPDATE, compute balanceAfter, write ledger + level
}
module.exports = { applyMovement /* , ... */ };
```

3.6 Route and mount

```
const router = require('express').Router({ mergeParams: true });
const may = (action) => requirePermission('warehouse', action);
router.get('/', may('canView'), validate(schema.list), controller.list);
/** Before `/:id`, or "reorder" would be read as an id. */
router.patch('/reorder', may('canEdit'), validate(schema.reorder), controller.reorder);
router.post('/', may('canCreate'), validate(schema.create), controller.create);
router.put('/:id', may('canEdit'), validate(schema.update), controller.update);
router.patch('/:id/status', may('canEdit'), validate(master.statusBody), controller.toggleStatus);
router.delete('/:id', may('canDelete'), validate(master.idParam), controller.remove);
module.exports = router;
```

Order: guard → `validate` → handler. Literal paths (`/summary` , `/reorder` , `/analytics` , `/quota`) before `/:id` . Mount tenant routes in `routes/myCompany.routes.js` (under `/admin/company`) or `modules/admin/admin.routes.js` ; super-admin in `modules/superadmin` ; public in `modules/theme` .

3.7 Menus, permissions, Swagger

- Add `{ name, slug, icon, route, sequence, parent? }` to `seeders/defaultMenus.js` ; add to `MENU_INHERITS_PARENT` if it is part of its parent's screen; to `MENU_FUNCTIONALITY` if plan-gated. Boot inserts it and grants every Company Admin role.
- New upload folder → `FOLDERS` in `middlewares/upload.js` .
- Swagger: routes are auto-scanned; add `@openapi` prose blocks and register request schemas in `SCHEMA_SOURCES` (`src/docs/openapi.js`).
- Backfills: an idempotent `ensureX()` step in `seeders/bootstrap.js` .

4. Angular consoles

4.1 Conventions

Components	Standalone (no NgModule), `ChangeDetectionStrategy.OnPush` , `inject()` in fields, data loaded in the constructor (no `ngOnInit`).
State	`signal()` , `computed()` , `effect()` ; signal inputs/outputs (`input()` , `output()`); `.subscribe({ next, error })` .
Templates	Built-in control flow only: `@if` , `@for (x of xs; track x.id)` , `@empty` , `@switch` . Separate `.html` via `templateUrl` .
Forms	Reactive, `fb.nonNullable.group` ; `<app-field-error>` ; `touchAll(form)` on invalid submit; `cleanPayload()` .
Styles	Global design system in `src/styles.scss` (tokens + primitives: `.card .btn .table .badge .modal .field .input` …). Component styles inline `styles: [...]` using `var(--…)` only; budget 4 kB warn / 8 kB error.

HTTP	Only through `ApiService` (unwraps envelope). Errors: `toast.error('Could not …', messageOf(err))`.
Naming	`kebab-name.component.ts/.html`; `*-list`, `*-detail`, `*-manager` (Company Details editors); selectors `app-*`.
Models	`interface X extends AuditFields`; string-literal unions for enums; `as const` arrays.
Tests	None present; schematics skip spec files.

4.2 Typical list component

```
@Component({
  selector: 'app-supplier-list',
  changeDetection: ChangeDetectionStrategy.OnPush,
  imports: [ReactiveFormsModule, PageHeaderComponent, PagerComponent, StatusBadgeComponent,
            TableStateComponent, ModalComponent, FieldErrorComponent, CanDirective],
  templateUrl: './supplier-list.component.html',
})
export class SupplierListComponent {
  private readonly companies = inject(CompanyService);
  private readonly toast = inject(ToastService);
  private readonly confirm = inject(ConfirmService);
  readonly store = new ListStore<Supplier>((q) => this.companies.listSuppliers(q));
  readonly modalOpen = signal(false);
  readonly saving = signal(false);
  constructor() { this.store.reload(); }
}
```

Template skeleton: `<app-page-header>` with actions → `.card` → search input → `<app-table-state>` → `.table-wrap > table.table` with `th.sortable` → `<app-pager>` → `@if (modalOpen()) { <app-modal> … <ng-container footer> }`. Write buttons wrapped in `*appCan="'suppliers'; action: 'canCreate'"` (Admin only).

4.3 Adding a screen

Step	Super Admin	Company Admin
Model	`core/models/domain.model.ts` (or a new `x.model.ts`)	
Service	Method in a `core/services/*.service.ts` calling `/super-admin/…`; masters via `MASTER_PATHS` + `CrudFactory`	Section in `company.service.ts` or a dedicated service with a `BASE` constant; card lists via `cards<T>()`
Component	Copy `features/masters/state-list.*` (CrudPage)	Copy `features/roles/role-list.*` (ListStore + appCan)
Route	Child of shell in `app.routes.ts`, title `'X · Aj Smart Biz'`	Child with `canActivate: [permissionGuard('slug')]`, title = page name

Step	Super Admin	Company Admin
Menu	Add to `navGroups` in `layout/shell.component.ts`	Seed in backend `defaultMenus.js` (sidebar comes from `/auth/me`); optional emoji in `ICONS`
Gates	—	`<app-feature-gate>` for plan features; `<app-limit-notice>` for quotas

5. Next.js websites

Rendering	Server Components by default; `'use client'` only for interactivity; client components receive props (themes pass `siteCompany(company)`).
Data	Themes: `getCompanyDetails()` (`lib/company.server.ts`, no-store) + pure `resolveX(company)` view-models in `lib/x.ts`. Technology: only `lib/api.ts` (`getSite()`, revalidate 300).
Server-only	`*.server.ts` with `import 'server-only'`.
Writes	Server Actions in `app/actions/<verb>-<noun>.ts` (`'use server'`), using `API_URL` and `domain: await resolveDomain()`, returning `{ status, message, fieldErrors? }`; types in a sibling `lib/<noun>-submit.ts`; forms use `useActionState` / `useFormStatus`.
Page guard	`apiReachable` → `resolved` → `service.active` → feature or `notFound()`; metadata via `anonymousMetadata(company) ?? {...}`.
CSS	`globals.css` tokens (`--brand --accent --surface --radius* --shadow*`) + co-located `Name.module.css`; tokens only, tints with `color-mix`; no Tailwind.
Naming	Components PascalCase (`ProductsSection.tsx`), page bands end in `Section`; lib files kebab-case; config constants SCREAMING_SNAKE; imports via `@/`.
Sections	Return `null` when their feature/content is absent — pages carry no conditions.

6. Recipe: add a new feature end to end

Example: a sellable “Suppliers” feature for the company workspace that also shows on the website.

1. **Constants** — add `FUNCTIONALITY.SUPPLIERS` + `FUNCTIONALITY_CATALOGUE` entry; any enums as `X` / `X_VALUES`.
2. **Model** — `models/companySupplier.model.js`; register and associate in `models/index.js`.
3. **Validator** — `validators/supplier.validator.js` (create/update/list).
4. **Service** (if rules) — `services/supplier.service.js`.
5. **Controller** — `controllers/supplier.controller.js` with `resolveCompanyId`, `assertGranted` on writes.

6. **Routes** — `routes/supplier.routes.js`; mount under `/admin/company/suppliers` with `requirePermission('suppliers')`; super-admin nested mount if needed.
7. **Menu** — `defaultMenus.js` row + `MENU_FUNCTIONALITY`.
8. **Public payload** (if on the website) — add `features.suppliers` in `functionality.service` / `public.controller`, `null` unless active; add a `NAV_PAGES` row if it is a page.
9. **Swagger** — `@openapi` blocks + `SCHEMA_SOURCES`.
10. **Admin UI** — model, service, `features/suppliers/supplier-list.component.*`, route with `permissionGuard`, `*appCan`, `<app-feature-gate>`.
11. **Super Admin UI** — nothing for the plan form (it reads the catalogue); add screens only if operators need them.
12. **Themes ×3** — `SiteFeatures` type + `FALLBACK_COMPANY` null; `lib/suppliers.ts` resolver; `SuppliersSection.tsx` + per-theme `.module.css`; page with guard chain.
13. **Verify** — run the API on SQLite, exercise endpoints (Swagger), `ng build` both consoles, `npm run typecheck` + `build` each theme.
14. **Document** — README section, `docs/schema.dbml`, these docs.

7. Formatting, naming and comments

Area	Rule
Indentation	2 spaces, UTF-8, final newline, trim trailing whitespace (`.editorconfig`).
Quotes / width	Single quotes; Prettier `printWidth: 100` in Angular apps (HTML with angular parser).
Backend files	camelCase with suffix: `.model.js .validator.js .controller.js .service.js .routes.js`; `'use strict'` first line.
Database	Model PascalCase singular; table snake_case plural, `company_` prefix for tenant tables; columns snake_case (raw SQL uses snake_case).
Keys	Menu slugs kebab-case; functionality keys snake_case; constants UPPER_SNAKE.
Comments	JSDoc paragraphs explaining *why*; section dividers `/* ---- name ---- */`; route-order notes.
Messages	User-facing errors are full sentences the UI can show verbatim.

8. Definition of done / review checklist

☐	Check
☐	No hard-coded tenant data in any client; new data comes from an endpoint
☐	Company taken from token; queries scoped; foreign ids 404

☐	Check
☐	Writes validated with Joi; update schemas have no defaults
☐	Permission slug enforced on API and mirrored in UI
☐	Plan-gated features use `assertGranted` and appear as `null` when not live
☐	Multi-step writes in one transaction; history snapshotted
☐	Literal routes declared before `/:id`
☐	Menu seeded; Swagger prose and schemas added
☐	Theme changes applied to all three designs
☐	Both consoles build; themes typecheck and build; README/docs updated